Layout Validation Rules

All 36 rules in `src/core/rules.ts` — why each exists, and how it's computed.

Core concepts

The graph. Every rule reads one graph built by `computeDwellingGraph()` in `adjacencyGraph.ts` from all floors at once. Nodes (`GraphNode`) are rooms, merged circulation/outdoor clusters, and stairs, each carrying `kind`, `roomTypeId`, `floor`, `cells`, and two fields other systems derive and rules just read: `hasExteriorEdge` and an optional `glazing` stat.

Two edge types. `GraphEdge.viaDoor: boolean` is the whole model. A **touch** edge (`viaDoor: false`) means two spaces share a wall. An **access** edge (`viaDoor: true`) means an authored door joins them. Reachability rules (the H-family, ST2, G1, C1/C2, O1, DP1, PG1, F1) traverse access edges only — a shared wall with no door is not a connection. Character/proximity rules (S3, AC1, H4, S6, S5) read whichever edge type matches what they're describing: a physical fact reads touch, a circulation fact reads access.

Entrances as roots. `graph.entryIds` holds the host-node id of every non-blocked entrance (ground floor only, re-derived on every graph build — nothing is cached). `ctx.hasEntrance = entryIds.length > 0`. Every reachability rule checks this first and returns `[]` if it's false; E1 covers the no-entrance case with its own note instead.

The RuleContext. `buildContext(graph)` runs once per `validate()` call and builds `ctx`, which every rule's `check(graph, ctx)` receives: `ctx.adj` (an access-only adjacency map — built by iterating `graph.edges` and adding both directions only where `e.viaDoor` is true), `ctx.viaStairAdj` (the same, but only edges where `e.viaStair` is also true — isolates a stair's floor-above connections from its same-floor ones), `ctx.degree(id)` (= `ctx.adj.get(id)?.size`, i.e. door count), `ctx.reachableFrom(seeds, blocked?)` (the BFS primitive described below), and `ctx.is.*` — type predicates (`bedroom`, `bathroom`, `kitchen`, `living`, `stair`, `outdoor`, `circulation`, `roomOrStair`, `public`, `habitable`) built from simple `roomTypeId` / `kind` matches.

The blocked-BFS pattern. `ctx.reachableFrom(seeds, blocked?)` is a multi-source BFS over `ctx.adj`. Seeds are always entered regardless of `blocked`; during the walk, any neighbour for which `blocked(node)` is true is never added to the visited set or queue. Five rules (H2, H3, H6, G1, S7) call this twice — once with no `blocked` argument (the full reachable set) and once with one room type blocked as an intermediate — then compare the two sets. A node reachable in the first call but not the second can only be reached by crossing the blocked type.

The `edgeViolations()` helper. Five rules (H4, S6, S3, AC1, S5) are pure edge-matching checks and share one function: `edgeViolations(graph, ctx, ruleId, severity, matchFn, access = false, excludeAccessCovered = false)`. It filters `graph.edges` by `viaDoor` matching the `access` flag, resolves each edge's two endpoint nodes via `ctx.nodesById`, and applies `matchFn(a, b)` (usually built with the small symmetric helper `pair(p, q, a, b) = (p(a)&&q(b)) || (p(b)&&q(a))`, so edge direction doesn't matter). When `excludeAccessCovered` is set, it first builds a `Set` of every access-edge pair (keyed by the order-independent `pairKey(a, b)`) and skips any touch edge that's also in it — the exact mechanism that keeps H4 (door) and S6 (wall) from ever both firing on the same boundary.

The depth/hop machinery. `accessDepths(graph, seeds)` is a 0-1 BFS (array-as-deque) over access edges, independent of `ctx.adj`. Its weighting: entering a stair node costs 1 hop, leaving one costs 0, everything else costs 1 — so a floor transition costs exactly one hop in total, not two. `computeEntranceDepths(graph)` is just

`accessDepths(graph, graph.entryIds)`. DP1, PG1, and F1 all build on this one function (F1 with a different seed list).

Severity tiers. Every rule carries `severity: "hard" | "soft" | "note"`. Hard renders the dwelling uninhabitable or breaks near-universal code. Soft departs from empirical practice or comfort norms. Note characterizes a recognized pattern without judging it. `validate(graph)` runs all 35 `RULES` entries and concatenates every `Violation` they return; nothing here ever blocks placement — it only runs on "Check Layout" and only reports.

Hard (11) — D1, E2, H1, H2, H3, H4, H6, OD1, P1, P2, ST2

Soft (19) — A1, AC1, C1, C2, D2, DP1, F1, G1, G2, MB1, N1, O1, OR1, PG1, S1, S2, S3, ST1, W1

Note (7) — DR1, DR2, E1, P3, S5, S6, S7

The rules

E1 Entrance required to validate

NOTE

WHY Reachability is measured from the entrance; with none placed there's nothing to measure from, so this one note replaces the reachability flags instead of leaving them silently absent.

CODE `check(graph, ctx)` returns `[]` if `ctx.hasEntrance` is true (set in `buildContext()` as `graph.entryIds.length > 0`). The description branches on `graph.entrances.length > 0` — entrances exist but are all blocked vs. none were ever placed — picking between a custom string and the static `RULES_BY_ID.E1.description`.

E2 Entrance blocked

HARD

WHY An entrance whose edge got built over no longer opens to the outside.

CODE `graph.entrances.filter(e => e.blocked).map(...)` — one violation per blocked entrance, carrying `entranceIds: [e.id]` and `nodeIds: [e.hostId]`. The `blocked` flag isn't computed in `rules.ts` at all — it lives on `EntranceStatus` in `adjacencyGraph.ts`, re-tested against `exteriorEdges()` on every graph build, so a room built later against the entrance invalidates it with zero rules-side bookkeeping.

DR1 No doors placed

NOTE

WHY Reachability is door-based, so a plan with rooms and zero doors floods H1 on every room; this note explains why before that flood looks like a bug.

CODE `if (graph.doorCount > 0) return [];` then `if (!graph.nodes.some(n => ctx.is.room(n))) return [];` (nothing to explain if there are no rooms yet either). `doorCount` is a field on `DwellingGraph`, the total authored-door count across all floors, computed in `adjacencyGraph.ts`.

DR2 Over-doored bedroom

NOTE

WHY A bedroom with three or more doors is unusual — most carry one, sometimes two for an en-suite — and starts costing furnishable wall space and privacy.

CODE `graph.nodes.filter(n => ctx.is.bedroom(n) && ctx.degree(n.id) >= 3)`. The message is built per-node with the live count interpolated (`` `Bedroom has ${ctx.degree(n.id)} doors...` ``) rather than the static `RULES_BY_ID` string, since the number varies.

P1 Bathroom required

HARD

WHY Basic program requirement — a dwelling with no sanitary room isn't habitable.

CODE `graph.nodes.some(ctx.is.bathroom)` — one boolean across the whole graph, independent of floor or reachability. `ctx.is.bathroom` matches `roomTypeId` against `["bathroom_small", "bathroom_large"]`.

P2 Kitchen required

HARD

WHY Basic program requirement.

CODE Identical shape to P1, with `ctx.is.kitchen` (`roomTypeId === "kitchen"`).

P3 More than one kitchen

NOTE

WHY Unusual for a single dwelling, not wrong (shared house, granny flat).

CODE `graph.nodes.filter(ctx.is.kitchen).length <= 1` returns `[]`; otherwise one dwelling-level note with `layout: true`.

MB1 Sleeping floor without a bathroom

SOFT

WHY A sleeping floor with no toilet means a stair trip at night — a comfort issue, not a code failure, and it must never double up with P1's total-absence case.

CODE Guards first on `graph.nodes.some(ctx.is.bathroom)` — bails to `[]` entirely if no bathroom exists anywhere (P1 already owns that). Then iterates `new Set(graph.nodes.map(n => n.floor))`; per floor, filters `onFloor`, checks `beds = onFloor.filter(ctx.is.bedroom)` against `!onFloor.some(ctx.is.bathroom)`. One violation per offending floor, `nodeIds` set to that floor's bedroom ids.

H1 Orphaned room

HARD

WHY A room with no doored path from any entrance can't be used.

CODE `const reach = ctx.reachableFrom(ctx.entryIds)`, no `blocked` argument — the unconstrained multi-source BFS over `ctx.adj`. `graph.nodes.filter(n => ctx.is.room(n) && !reach.has(n.id))`. This is the simplest consumer of `reachableFrom`; every other reachability rule builds on the same call with a `blocked` predicate added.

H2 Reachable only through a bathroom

HARD

WHY The only doored route to this space crosses a bathroom — a privacy and practicality failure.

CODE `const full = ctx.reachableFrom(ctx.entryIds)` and `const noBath = ctx.reachableFrom(ctx.entryIds, ctx.is.bathroom)` — the second call passes `ctx.is.bathroom` as the `blocked` predicate, so bathroom nodes are never walked *through* during that BFS (they can still be entered as a seed or as a leaf, just not crossed). Filter: `ctx.is.roomOrStair(n) && !ctx.is.bathroom(n) && full.has(n.id) && !noBath.has(n.id)` — present in the unconstrained set, absent from the blocked one.

H3 Reachable only through a bedroom

HARD

WHY Same failure mode as H2 for bedrooms, except a bathroom reached only through its own bedroom is an en-suite, not a failure, so bathrooms are exempt from this one.

CODE Same two-`reachableFrom` pattern with `ctx.is.bedroom` as the blocked predicate. The filter adds `&& !ctx.is.bathroom(n)` on top of `&& !ctx.is.bedroom(n)`, so a bathroom that only survives via a bedroom-crossing path is deliberately excluded here — S7 flags exactly that excluded case, positively.

H4 Direct bathroom–kitchen door

HARD

WHY A door straight from food preparation into a toilet is a hygiene hazard.

CODE `edgeViolations(graph, ctx, "H4", "hard", (a,b) => pair(ctx.is.bathroom, ctx.is.kitchen, a, b), true)`. The trailing `true` is the `access` parameter — `edgeViolations` keeps only edges where `e.viaDoor === true`, so H4 never sees a sealed shared wall at all.

S6 Shared wet wall

NOTE

WHY The positive counterpart to H4: a kitchen and bathroom sharing a wall with no door is efficient plumbing (stacked riser), worth confirming rather than flagging.

CODE `edgeViolations(graph, ctx, "S6", "note", pair(ctx.is.bathroom, ctx.is.kitchen), false, true)` — `access=false` reads touch edges; the trailing `true` is `excludeAccessCovered`, which builds `accessPairs = new Set(graph.edges.filter(e => e.viaDoor).map(e => pairKey(e.a, e.b)))` and skips any touch pair also present there. This is the exact mechanism that keeps H4 and S6 from co-firing on one boundary.

H6 Reachable only through outdoor space

HARD

WHY A room reachable only by crossing a balcony or terrace is weather-dependent before anyone can use it.

CODE Same two-`reachableFrom` pattern, `ctx.is.outdoor` blocked. No extra exclusion needed in the filter (unlike H2/H3) because `ctx.is.roomOrStair` only matches `kind === "room" || kind === "stair"`, and outdoor spaces are `kind: "cluster"` — structurally outside the target set already.

C1 Orphaned corridor

SOFT

WHY A circulation cluster with no doors onto it is dead space — a design flaw, not uninhabitability, hence soft (demoted from an earlier hard version).

CODE `graph.nodes.filter(n => ctx.is.circulation(n) && ctx.degree(n.id) === 0)`. `ctx.is.circulation` matches `kind === "cluster" && roomTypeId === "circulation"`.

C2 Under-used corridor

SOFT

WHY A corridor with exactly one door doesn't circulate — you step in and back out.

CODE Identical to C1's filter with `ctx.degree(n.id) === 1`.

A1 Circulation below accessible width

SOFT

WHY SIA 500 wants ~1.2 m clear width for accessibility (2 grid cells); nothing else in the ruleset checks width, so a plan could have 600 mm corridors with 1200 mm doors opening onto them.

CODE Iterates circulation nodes and calls the module-level helper `narrowWidthCells(n.cells)`, defined outside the rules table. It builds a `Set` of `"cx,cz"` keys for the cluster's cells, then for every cell tests the four candidate 2×2 blocks that could contain it (offsets `dx,dz ∈ {-1,0}`) via `fits2x2(mx,mz)`, which checks all four corners of that block are present in the set. A cell is narrow if none of the four blocks is fully occupied by the cluster. The message interpolates `narrow.length`; `nodeIds` is just the cluster id — narrow cells aren't individually highlighted.

O1 Unconnected outdoor space

SOFT

WHY A balcony or terrace nothing opens onto can't be used — the outdoor mirror of C1. Since the semi-exterior pass "connected" no longer means "doored": a french window counts. Gated to the pre-entrance case, because once the dwelling has an entrance OD1 owns the same node with a stronger claim.

CODE `!ctx.hasEntrance`, then `graph.nodes.filter(n => ctx.is.outdoor(n) && ctx.degree(n.id) === 0)`.

OD1 Outdoor space not reachable

HARD

WHY A balcony you cannot get to is floor area the dwelling cannot use. With french-window access this fires only when the balcony touches no room along a run wide enough to glaze — a 1-cell contact is 0.6 m of wall, not a way through — or when the only rooms it touches are themselves unreachable from the entrance.

CODE `ctx.reachableFrom(ctx.entryIds)`, then outdoor nodes not in the set. Outdoor clusters are routing *leaves*: reachable, never a through-route between two interior rooms.

ST1 Stair unconnected at an end

SOFT

WHY A stair should be doored at both the floor it leaves and the floor it arrives at.

CODE Per stair node: `const top = ctx.viaStairAdj.get(n.id)?.size ?? 0` and `const bottom = ctx.degree(n.id) - top`. `ctx.viaStairAdj` is built alongside `ctx.adj` in `buildContext()` — same door-edge iteration, but only edges where `e.viaStair` is also true get added, isolating the floor-above side from the same-floor side. `missing` collects `"bottom"` / `"top"` by name into the message.

ST2 Stair unreachable

HARD

WHY An unreachable stair cuts off whatever floor it serves — H1 for vertical circulation.

CODE Same reachable-set check as H1 (`ctx.reachableFrom(ctx.entryIds)`, no blocking), filtered to `ctx.is.stair(n)` instead of `ctx.is.room(n)`.

D1 No exterior wall — no daylight

HARD

WHY A habitable room with no exterior wall gets no daylight; no interior arrangement fixes that.

CODE `graph.nodes.filter(n => ctx.is.habitable(n) && !n.hasExteriorEdge)`. `ctx.is.habitable` = `bedroom(n) || isPublic(n)` (living/recreation). `hasExteriorEdge` is **NOT** computed in `rules.ts` — it's a `GraphNode` boolean set once per floor in `adjacencyGraph.ts`, derived from the same `buildSpaceTargets()` occupied-cell set the door system uses, which is what makes it correctly exclude edges facing a stairwell hole.

D2 Kitchen without ventilation

SOFT

WHY Same check for kitchens, soft not hard — internal kitchens are common and get mechanical ventilation.

CODE `graph.nodes.filter(n => ctx.is.kitchen(n) && !n.hasExteriorEdge)` — same field as D1, different predicate and tier.

W1 Glazing below daylight target

SOFT

WHY A room's actual glazing area falls short of its type's target, even though it does have exterior wall.

CODE `graph.nodes.filter(n => n.hasExteriorEdge && n.glazing?.belowTarget)`. `node.glazing` is a `GlazingStat` populated from `floor.windowStats`, itself the output of `computeWindows()` in `windows.ts` — W1 never recomputes windows, it just reads the flag. That function looks up `WINDOW_CONFIG[roomTypeId]` (living/recreation `1/6` full-height; bedroom/kitchen `1/10` framed; kitchen additionally fixed at `fixedEdges: 2`), computes `floorArea = cells.length * 0.36`, computes `perEdge = perEdgeGlazing(variant, floorHeight)` (`0.6 * (floorHeight - 0.9)` full-height, minus another `0.9` for a framed lintel), derives `edgesNeeded = ceil(floorArea * targetRatio / perEdge)` clamped up to `MIN_WINDOW_EDGES = 2`, then bands the exterior runs SOUTHERNMOST-first (under the project north; length as the tie-break). `belowTarget = placed < targetEdges`. The `hasExteriorEdge` guard keeps W1 from double-firing on a room D1/D2 already own.

OR1 Lit only from the north

SOFT

WHY A habitable room or kitchen whose every window faces north gets no meaningful direct sun. Solar-access practice at this latitude wants living-hours rooms glazed toward the south; a purely northern room is worth flagging — as a heuristic, not a code failure.

CODE `graph.nodes.filter(n => (ctx.is.habitable(n) || ctx.is.kitchen(n)) && n.glazing?.northLit)`. `northLit` is derived in `computeWindows()` (which now takes `northAngle`): each windowed edge's side becomes a compass bearing via `orientation.ts` (convention: north is world −Z rotated clockwise, viewed from above, by `northAngle`), and the flag is true only when the room HAS glazing AND every windowed edge lies within `NORTH_SECTOR_HALF_WIDTH = 45°` of due north. Because a no-glazing room is `northLit === false`, OR1 can never double-fire with D1/W1 (which own the no-daylight cases). The generator biases glazing toward south, so OR1 only fires when a room's sole exterior walls face north.

G1 Guest access to a bathroom

SOFT

WHY A guest should be able to reach a toilet without crossing someone's bedroom — a dwelling-wide question, not a per-room one.

CODE Guarded first on `bathrooms.length === 0` (bail to `[]`; P1 owns total absence). Then `const noBed = ctx.reachableFrom(ctx.entryIds, ctx.is.bedroom)` and checks `bathrooms.some(n => noBed.has(n.id))`. If none survive, one dwelling-level violation (`layout: true`, empty `nodeIds`) fires.

G2 Entrance into a private room

SOFT

WHY The front door opening straight into a bedroom or bathroom skips the usual public threshold — deliberate in studios, hence soft.

CODE Iterates `graph.entrances` directly (not the reachability graph); for each non-blocked entrance with a `hostId`, looks up the host node and tests `ctx.is.bedroom(host) || ctx.is.bathroom(host)`. Carries both `nodeIds: [host.id]` and `entranceIds: [e.id]` so both the room and the marker highlight.

S1 Over-connected outdoor space

SOFT

WHY An outdoor space with more than two doors onto it is usually being used as a through-route, not a leaf.

CODE `graph.nodes.filter(n => ctx.is.outdoor(n) && ctx.degree(n.id) > 2)`.

S2 Under-connected living room

SOFT

WHY A living room reached by one door or none isn't functioning as the social hub it's meant to be.

CODE `graph.nodes.filter(n => ctx.is.living(n) && ctx.degree(n.id) <= 1)`.

S3 Bedroom beside a social room or kitchen

SOFT

WHY A bedroom directly touching a kitchen, living room or recreation room bleeds noise and compromises privacy.

CODE `edgeViolations(graph, ctx, "S3", "soft", (a,b) => pair(ctx.is.bedroom, n => ctx.is.kitchen(n) || ctx.is.public(n), a, b))` — called with no `access` / `excludeAccessCovered` args, so it defaults to touch edges with no dedup against doored pairs; fires whether or not there's also a door on that wall.

AC1 Bedroom beside a stair

SOFT

WHY Stairs carry impact and airborne noise (SIA 181); replaced an earlier rule (two bedrooms touching) that had no defensible grounding.

CODE `edgeViolations(graph, ctx, "AC1", "soft", pair(ctx.is.bedroom, ctx.is.stair))` — same touch-edge default as S3, scoped to stairs specifically so it never overlaps S3 (bedroom-vs-recreation is S3's case, not AC1's).

S5 Open-plan kitchen–living

NOTE

WHY A doored kitchen–living connection reads as open-plan — confirmed, not flagged.

CODE `edgeViolations(graph, ctx, "S5", "note", pair(ctx.is.kitchen, ctx.is.living), true)` — `access=true` reads door edges only; a sealed touching wall earns nothing.

S7 En-suite bathroom

NOTE

WHY A bathroom reachable only through its own bedroom is an en-suite — the exact case H3 exempts, named positively here.

CODE Recomputes its own `full` / `noBed` pair (independent call, not shared with H3's) and filters `ctx.is.bathroom(n) && full.has(n.id) && !noBed.has(n.id)` — the mirror image of H3's exclusion clause.

DP1 Unusually deep room

SOFT

WHY A room five or more doored hops from the entrance is unusually buried in the plan (space-syntax depth).

CODE `const depths = computeEntranceDepths(graph)`, then flags rooms with `d >= DEEP_ROOM_THRESHOLD_HOPS` (= 5). `computeEntranceDepths` is just `accessDepths(graph, graph.entryIds)` — a 0-1 BFS (array-as-deque) over an access-only adjacency map built independently of `ctx.adj`. Weighting: `stepCost(from,to) = isStair.has(to) ? 1 : isStair.has(from) ? 0 : 1` — entering a stair costs 1, leaving one costs 0. Relaxations with weight 0 are `unshift`ed to the front of the deque (processed next), weight-1 relaxations are `push`ed to the back — standard 0-1 BFS.

N1 Circulation-heavy layout

SOFT

WHY Too much of the interior given over to circulation is inefficient — every circulation m² is cost without habitable benefit. Checked whole-dwelling AND per floor, since one bloated storey can otherwise hide behind efficient others in the average.

CODE `computeCirculationFraction(graph)` sums each node's `cells.length` into `denom`, skipping outdoor clusters entirely (`continue`, excluded from both sides), and additionally into `circ` when the node is a circulation cluster or a stair. Returns `circ/denom`, or `null` if `denom === 0`. The rule compares that against `CIRCULATION_FRACTION_MAX` (= 0.25). `computeCirculationFractionByFloor(graph)` shares the same per-node tally (factored into `circulationCellCounts()`) but keyed by `GraphNode.floor`; on a multi-floor dwelling (`graph.floorCount > 1`) the rule additionally flags any floor whose own fraction crosses the same threshold, naming the floor and pointing at that floor's circulation/stair nodes — suppressed on a single floor, where it would just repeat the whole-dwelling figure. Both functions back the report's always-on "Circulation: N%" line and its per-floor breakdown line in `validationPanel.ts`, so the flags and the printed numbers can never disagree.

PG1 Inverted privacy gradient

SOFT

WHY Hillier & Hanson's genotype expects public rooms shallower than bedrooms; this flags the inversion.

CODE `publicVsBedroomDepth(graph, depths)` — `depths` comes from a fresh call to `computeEntranceDepths(graph)`, not shared with DP1's. It computes `meanOf(pred)` twice, once for `roomTypeId ∈ {living, recreation}` and once for `{bedroom_small, bedroom_large}`, filtering to nodes that actually have a depth entry (i.e. are reachable) before averaging. Returns `null` if either set is empty. Fires when `publicMean > bedroomMean`; both means also print in the report regardless.

F1 Far from an exit

SOFT

WHY A room far from any exit is an egress concern — approximated as a hop count, not fire-code metres of travel distance.

CODE `const stairIds = graph.nodes.filter(ctx.is.stair).map(n => n.id)`, then `accessDepths(graph, [...ctx.entryIds, ...stairIds])` — the exact same 0-1 BFS function DP1 uses, called with a different seed list: entrances *and* every stair, instead of entrances alone. Flags rooms with `d > ESCAPE_DEPTH_MAX` (= 4). Because it's the same function as DP1 with only the seeds changed, the two rules can diverge sharply on multi-floor plans, where a room sits deep from the entrance but close to a stair.

Tier counts: 10 hard, 18 soft, 7 note. Written from `src/core/rules.ts` and `src/core/windows.ts`; every id, tier, and constant here matches the code as built.